

Aprendizagem por Reforço - Aulas Práticas

Índice

1. Blackjack Environment ([blackjack.py](#))
 2. Windy Gridworld Environment ([windy_gridworld.py](#))
 3. Tabular SARSA Agent ([sarsa.py](#))
 4. Semi-Gradient SARSA com PyTorch ([torch_sarsa.py](#))
 5. Semi-Gradient SARSA com NumPy ([linear_sarsa.py](#))
 6. Ambiente de Jogo Tic-Tac-Toe ([tictactoe.py - envs](#))
 7. Codificação de Estado para Tic-Tac-Toe ([tictactoe.py - Features](#))
 8. Seleção de Políticas no Tic-Tac-Toe ([tictactoe.py - Experiments](#))
 9. REINFORCE Agent ([reinforce.py](#))
 10. Self-Play e Penalização no REINFORCE ([reinforce_tictactoe.py](#))
 11. Monte Carlo Tree Search - MCTS ([mcts.py](#))
 12. Multi-Armed Bandits ([kbandits_incomplete.py](#))
 13. Programação Dinâmica com Gridworld ([TP2_MDP_GridWorld.ipynb](#) & [Practical3_Gridworld_INCOMPLETE.ipynb](#))
 14. Jack's Car Rental ([Practical3_CarRental_INCOMPLETE.ipynb](#))
-

1. Blackjack Environment ([blackjack.py](#))

Ficheiro

[[blackjack.py](#)](file:/AR_MIA/mia_rl/envs/blackjack.py)

Descrição do TODO

Implementar a lógica de transição para a ação "[hit](#)" (pedir carta) no ambiente de Blackjack:

1. Comprar (dar) uma nova carta ao jogador.
2. Calcular o estado resultante ([next_state](#)).
3. Se o jogador ultrapassar 21 pontos (*bust*), terminar o episódio com recompensa -1.0 .
4. Caso contrário, continuar o jogo com recompensa 0.0 .

Código Implementado

```
if action == "hit":
    self.player.append(draw_card(self.rng))
    next_state = self._state()
    if is_bust(self.player):
        return next_state, -1.0, True
    return next_state, 0.0, False
```

Detalhes

- **Representação do Estado (S):** O estado é um tuplo (`soma_jogador`, `carta_visivel_dealer`, `tem_as_utilizavel`).
- **Atualização da Mão:** A função `draw_card(self.rng)` escolhe aleatoriamente uma carta do baralho `DECK = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10]`, onde as cartas de figuras (Valete, Dama, Rei) têm valor 10 e o Ás tem valor inicial 1.
- **Cálculo da Pontuação:** A função `sum_hand` calcula a soma total. Se houver um Ás na mão e a soma total incluindo o Ás como 11 não ultrapassar 21, o Ás é considerado "utilizável" e adicionam-se 10 pontos à soma (uma vez que o Ás já conta como 1 na soma base).
- **Condição de Bust:** Se `sum_hand(self.player) > 21`, o jogador perde automaticamente (*bust*). A recompensa de transição é $R_t = -1.0$ e o sinal de terminação é `done = True`.
- **Continuação do Jogo:** Se a soma for ≤ 21 , o estado transita para o novo estado calculado, com recompensa $R_t = 0.0$ e `done = False`, permitindo ao jogador tomar novas decisões no próximo passo.

2. Windy Gridworld Environment (`windy_gridworld.py`)

Ficheiro

[`windy_gridworld.py`](file:/AR_MIA/mia_rl/envs/windy_gridworld.py)

Descrição do TODO

Implementar a dinâmica de movimento do Windy Gridworld, considerando os efeitos do vento:

1. Ler o deslocamento básico (`delta_row`, `delta_col`) a partir do dicionário `ACTION_TO_DELTA`.
2. Aplicar a força do vento correspondente à coluna atual (`self.wind[col]`) para empurrar o agente para cima.
3. Garantir que as coordenadas resultantes não saem dos limites da grelha.
4. Retornar o novo estado, a recompensa de passo e o sinal de terminação (se alcançou o objetivo).

Código Implementado

```
row, col = state
delta_row, delta_col = ACTION_TO_DELTA[action]
wind_strength = self.wind[col]

next_row = min(max(row + delta_row - wind_strength, 0), self.rows - 1)
next_col = min(max(col + delta_col, 0), self.cols - 1)
next_state = (next_row, next_col)
done = next_state == self.goal
return next_state, self.reward_per_step, done
```

Detalhes

- **Representação do Espaço:** A grelha é indexada com coordenadas (`linha`, `coluna`) em que a linha 0 representa o topo e a coluna 0 a extrema esquerda.

- **Ações e Deslocamentos:** As ações são "up", "down", "left", "right". Os deslocamentos associados em linha e coluna são:

$$\Delta_{\text{up}} = (-1, 0), \quad \Delta_{\text{down}} = (1, 0), \quad \Delta_{\text{left}} = (0, -1), \quad \Delta_{\text{right}} = (0, 1)$$

- **Efeito do Vento:** O vento sopra verticalmente para cima e a sua força varia por coluna, definida em `self.wind`. Como o topo da grelha é a linha 0, empurrar o agente para cima equivale a *subtrair* a força do vento (`wind_strength`) da linha atual.
- **Equações de Transição:**

$$row_{next} = \max(0, \min(row + \Delta_{row} - \text{wind_strength}, \text{rows} - 1))$$

$$col_{next} = \max(0, \min(col + \Delta_{col}, \text{cols} - 1))$$

Isto impede que o agente saia dos limites físicos da grelha (grelha de dimensão $rows \times cols$).

- **Recompensa e Terminação:** A recompensa de cada passo é constante (`self.reward_per_step = -1.0`), incentivando o agente a encontrar o caminho mais curto. O episódio termina quando `next_state == goal` (neste caso, na coordenada (3, 7)).

3. Tabular SARSA Agent (`sarsa.py`)

O SARSA é um algoritmo de controlo por diferença temporal baseado em política ativa (*on-policy*). O agente aprende a função de valor de ação $Q(s, a)$.

Ficheiro

[sarsa.py](file:/AR_MIA/mia_rl/agents/control/sarsa.py)

3.1 Seleção de Ações (ϵ -greedy)

Descrição do TODO

Implementar a seleção de ações seguindo uma política ϵ -greedy, guardando a ação selecionada em cache.

Código Implementado

```
def select_action(self, state: StateT) -> ActionT:
    if self.rng.random() < self.epsilon:
        action = self.rng.choice(self.actions)
    else:
        best_value = max(self.action_value_of(state, action) for action in
self.actions)
        best_actions = [action for action in self.actions if
self.action_value_of(state, action) == best_value]
        action = self.rng.choice(best_actions)

    self._selected_actions[state] = action
    return action
```

Detalhes

- **Estratégia ϵ -greedy:**

- Com probabilidade ϵ (exploração), o agente escolhe uma ação aleatória da distribuição uniforme sobre todas as ações disponíveis $\mathcal{A}$:

$$a \sim \text{Uniforme}(\mathcal{A})$$

- Com probabilidade $1 - \epsilon$ (exploração/aproveitamento), o agente escolhe a ação com maior valor estimado:

$$a \in \operatorname{argmax}_{b \in \mathcal{A}} Q(s, b)$$

- **Desempate Aleatório (Tie-Breaking):** Em vez de utilizar apenas `max()` (que escolheria deterministicamente a primeira ação com o valor máximo, enviesando o comportamento do agente), o código calcula o valor máximo `best_value` e filtra todas as ações que alcançam esse valor. Em seguida, escolhe aleatoriamente uma dessas melhores ações através de `self.rng.choice(best_actions)`.
- **Necessidade de Cache:** Guardar a ação em `self._selected_actions[state]` é vital para o SARSA, porque as atualizações da função de valor utilizam a ação real que o agente de facto tomou no passo seguinte ($\mathcal{A}_{t+1}$).

3.2 Atualização Tabular SARSA

Descrição do TODO

Implementar a atualização temporal-diferencial (TD) clássica do SARSA.

Código Implementado

```
def update_transition(self, transition: Transition[StateT, ActionT]) ->
None:
    bootstrap = 0.0
    if not transition.done and transition.next_state is not None:
```

```

        next_action = self._selected_actions[transition.next_state]
        bootstrap = self.action_value_of(transition.next_state,
next_action)

        td_target = transition.reward + self.gamma * bootstrap
        current_value = self.action_value_of(transition.state,
transition.action)
        self.Q[(transition.state, transition.action)] = current_value +
self.alpha * (td_target - current_value)

```

Detalhes

- **Equação de Atualização do SARSA:**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- **Variáveis:**

- `transition.reward`: Recompensa imediata R_{t+1} .
- `self.gamma` (γ): Fator de desconto para recompensas futuras.
- `bootstrap`: O valor estimado do próximo estado-ação, $Q(S_{t+1}, A_{t+1})$. Se o estado for terminal (`transition.done == True`), não há futuro e o bootstrap é explicitamente definido como 0.0.
- `self.alpha` (α): Taxa de aprendizagem que controla a dimensão do passo de atualização.
- `td_target`: O alvo TD ($R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$).
- `td_target - current_value`: O erro de diferença temporal (Erro TD, δ_t).

4. Semi-Gradient SARSA com PyTorch (`torch_sarsa.py`)

Esta classe implementa o SARSA com aproximação linear de funções usando a biblioteca PyTorch.

Ficheiro

[`torch_sarsa.py`](file:/AR_MIA/mia_rl/agents/control/torch_sarsa.py)

Descrição do TODO

Completar o passo de atualização do otimizador de gradiente para semi-gradiente SARSA usando PyTorch.

Código Implementado

```

phi = self._to_tensor(transition.state, transition.action)
self.optimizer.zero_grad()
pred = self.model(phi)
target_tensor = torch.tensor([target], dtype=torch.float32)
loss = 0.5 * F.mse_loss(pred, target_tensor)
loss.backward()
self.optimizer.step()

```

```
delta = abs(target - pred.item())
self._td_errors.append(delta)
```

Detalhes

- **Aproximação de Função Linear:** O modelo é definido como um módulo linear sem bias:

$$\hat{q}(s, a; \mathbf{w}) = \mathbf{w}^T \phi(s, a)$$

onde `self.model` é `nn.Linear(self.n_features, 1, bias=False)`.

- **Semigradiente (Semi-Gradient):** Apenas os parâmetros do estado atual $\mathbf{w}$ recebem gradientes. O valor do alvo TD (que depende dos pesos no estado seguinte) é calculado numericamente e encapsulado num tensor isolado: `target_tensor = torch.tensor([target], dtype=torch.float32)` Este tensor não tem `requires_grad=True`, ou seja, o grafo de computação do autograd do PyTorch é interrompido no alvo, garantindo que o gradiente não flua de volta através da estimativa de bootstrap (o que define o método como *semi-gradiente* em vez de gradiente completo).
- **Ajuste da Função de Perda (Loss):** A perda quadrática média (MSE) padrão é multiplicada por 0.5 de forma deliberada. Aplicando a descida de gradiente estocástica (SGD) com taxa de aprendizagem α garante que a atualização efetuada pelo otimizador do PyTorch seja exatamente idêntica à regra analítica clássica.
- **Operações do Otimizador:**
 - `self.optimizer.step()`: Aplica a atualização de pesos pelo otimizador SGD.

5. Semi-Gradient SARSA com NumPy (`linear_sarsa.py`)

Esta classe implementa a aproximação de funções linear em NumPy, calculando analiticamente os gradientes e atualizando os pesos diretamente de forma manual.

Ficheiro

[`linear_sarsa.py`](file:/AR_MIA/mia_rl/agents/control/linear_sarsa.py)

Descrição do TODO

Implementar o passo de atualização analítica do semi-gradiente SARSA usando vetores NumPy.

Código Implementado

```
phi = self.phi(transition.state, transition.action)
if not transition.done and transition.next_state is not None:
    next_action = self._selected_actions[transition.next_state]
    bootstrap = self.action_value_of(transition.next_state, next_action)
else:
    bootstrap = 0.0
td_error = transition.reward + self.gamma * bootstrap - float(self.w @ phi)
```

```
self.w += self.alpha * td_error * phi
self._td_errors.append(abs(td_error))
```

Detalhes

- **Aproximação Linear:** A função de valor de ação aproximada é dada por:

$$\hat{q}(s, a; \mathbf{w}) = \mathbf{w}^T \phi(s, a)$$

onde $\mathbf{w}$ é o vetor de pesos `self.w` e $\phi(s, a)$ é o vetor de características (features) calculado por `self.phi(state, action)`.

- **Cálculo da Estimativa:** O produto interno entre os pesos e o vetor de características é calculado através do operador `@` do NumPy: `self.w @ phi`.
- **Atualização Semi-Gradiente:** A atualização direta do vetor de pesos é realizada por:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta_t \phi(S_t, A_t)$$

onde $\delta_t = R_{t+1} + \gamma \mathbf{w}^T \phi(S_{t+1}, A_{t+1}) - \mathbf{w}^T \phi(S_t, A_t)$ é o erro TD (`td_error`).

6. Ambiente de Jogo Tic-Tac-Toe (`tictactoe.py` - envs)

Ficheiro

[tictactoe.py (envs)](file:/AR_MIA/mia_rl/envs/tictactoe.py)

Descrição do TODO

Implementar o ambiente de Tic-Tac-Toe completando os seguintes métodos principais:

1. `reset()` — Reiniciar o tabuleiro e definir X (representado por `+1`) como o primeiro jogador.
2. `available_actions(state)` — Devolver uma lista com os índices (0 a 8) de todas as células vazias no estado atual.
3. `is_terminal(state)` — Devolver `True` se o jogo terminou por vitória de algum jogador ou por empate (tabuleiro cheio).
4. `step(action)` — Colocar a marca do jogador atual na célula especificada pela ação, calcular a recompensa para a jogada, alternar o turno do jogador e devolver a transição.
5. `render(state)` — Imprimir no terminal uma representação visual do tabuleiro, numerando as células vazias de 1 a 9 para facilitar a interação.

Código Implementado

- Método `reset()`:

```
self.board = (0,) * 9
self.current_player = 1
return self.board
```

- Método `available_actions(state)`:

```
return [index for index, cell in enumerate(state) if cell == 0]
```

- Método **is_terminal(state)**:

```
return _winner(state) != 0 or all(cell != 0 for cell in state)
```

- Método **step(action)**:

```
# Validação dos limites do tabuleiro
if action < 0 or action >= len(self.board):
    raise ValueError(f"Invalid action: {action}")
# Validação para assegurar que a célula escolhida está vazia
if self.board[action] != 0:
    raise ValueError(f"Cell {action} is already occupied.")

# Converte para lista para poder modificar a célula da ação selecionada
board_list = list(self.board)
board_list[action] = self.current_player
# Cria um novo tuplo imutável com o tabuleiro resultante
new_board = tuple(board_list)

# Identifica se a jogada resultou num vencedor
winner = _winner(new_board)
# O jogo termina se houver um vencedor ou empate (tabuleiro sem células a zero)
done = winner != 0 or all(cell != 0 for cell in new_board)
# A recompensa é de +1.0 para o jogador ativo caso tenha vencido
reward = 1.0 if winner == self.current_player else 0.0

# Atualiza o estado do ambiente e inverte o jogador ativo multiplicando por -1
self.board = new_board
self.current_player *= -1
return new_board, reward, done
```

- Método **render(state)**:

```
# Utiliza o estado atual do tabuleiro caso nenhum estado externo seja passado
board = self.board if state is None else state
symbols = {1: "X", -1: "O", 0: None}

# Representação de célula vazia numerada de 1 a 9 para guiar o jogador humano
def cell_repr(index: int) -> str:
    cell = board[index]
```



```

    if cell == 0:
        return str(index + 1)
    return symbols[cell]

# Desenha a grelha formatada de 3x3 no stdout
for row in range(3):
    start = row * 3
    print(" | ".join(cell_repr(start + col) for col in range(3)))
    if row < 2:
        print("----+----+----")

```

Detalhes

- **Representação do Tabuleiro:** O tabuleiro é armazenado internamente como um tuplo imutável de 9 elementos (`TicTacToeState`). Os elementos são: `1` para o jogador X, `-1` para o jogador O, e `0` para células vazias.
- **Linhas de Vitória:** A verificação de vitória (`_winner`) itera por um conjunto estático de 8 linhas vencedoras (3 horizontais, 3 verticais e 2 diagonais). Se a soma de elementos em qualquer uma destas linhas for `+3` (vitória de X) ou `-3` (vitória de O), retorna o respetivo jogador.
- **Mecanismo de Turnos:** O jogador ativo `self.current_player` é atualizado multiplicando por `-1` a cada passo. Isto alterna de forma simples e eficiente entre `+1` e `-1`.
- **Imutabilidade e Segurança de Estados:** Como o estado é um tuplo do Python, qualquer transição requer a criação de um novo tuplo. O método `step()` converte temporariamente o tuplo para lista (`list(self.board)`), insere a marca no índice correspondente, e converte-o de volta para tuplo (`new_board`), garantindo a preservação histórica de estados anteriores do ambiente (fundamental para rollouts de planeamento e cópias).
- **Recompensa do Passo:** A recompensa de transição em `step` é `1.0` se o jogador que acabou de mover vencer a partida, e `0.0` em caso de empate ou jogo em curso. A penalização do perdedor é tratada na injeção posterior na trajetória de self-play.

7. Codificação de Estado para Tic-Tac-Toe (`tictactoe.py` - Features)

Para aplicar aproximação de funções no Tic-Tac-Toe, é necessário converter a representação do tabuleiro num vetor numérico de características (features).

Ficheiro

[tictactoe.py (Features)](file:/AR_MIA/mia_rl/features/tictactoe.py)

Descrição do TODO

Criar uma codificação *one-hot* de 27 dimensões para o estado do tabuleiro de Tic-Tac-Toe, indexando as peças a partir da perspetiva do jogador atual.

Código Implementado

```
phi = np.zeros(STATE_FEATURE_DIM, dtype=np.float32)
for i, cell in enumerate(board):
    if cell == current_player:
        phi[i * 3 + 0] = 1.0 # my piece
    elif cell == -current_player:
        phi[i * 3 + 1] = 1.0 # opponent's piece
    else:
        phi[i * 3 + 2] = 1.0 # empty
return phi
```

Detalhes

- **Dimensão do Vetor ($D = 27$):** O tabuleiro tem 9 células. Cada célula é codificada por um subvetor *one-hot* de 3 dimensões:

Subvetor = [A minha peça, A peça do Oponente, Vazia]

Logo, a dimensão total é $9 \times 3 = 27$ características.

- **Mapeamento de Índices:** Para a célula $i \in \{0, \dots, 8\}$, os três slots correspondentes no vetor global ϕ são:
 - Peça do próprio jogador: slot $i \times 3 + 0$
 - Peça do oponente: slot $i \times 3 + 1$
 - Célula vazia: slot $i \times 3 + 2$
- **Decisão de Design:** O tabuleiro armazena tradicionalmente $+1$ para o jogador X e -1 para o jogador O. No entanto, ao codificar as posições relativas ao jogador cuja vez se joga (`current_player`), o estado é sempre interpretado de forma idêntica por ambos. X vê as suas peças no slot 0 e as de O no slot 1; O vê as suas peças no slot 0 e as de X no slot 1.

[!IMPORTANT] Esta representação relativa à perspetiva permite que um **único conjunto de pesos de política (ou rede neuronal)** seja partilhado por ambos os lados durante o treino por *self-play*. Isto duplica a eficiência de amostragem de dados e garante a simetria da aprendizagem.

8. Seleção de Políticas no Tic-Tac-Toe ([tictactoe.py](#) - Experiments)

Ficheiro

[tictactoe.py (Experiments)](file:/AR_MIA/mia_rl/experiments/tictactoe.py)

Descrição do TODO

Selecionar dinamicamente a política correta com base no jogador ativo do ambiente e recolher a respetiva ação.

Código Implementado

```
policy = policy_x if env.current_player == 1 else policy_o
action = policy(env, state)
```

Detalhes

- Durante a execução do ciclo de jogo, `env.current_player` alterna entre `+1` (jogador X) e `-1` (jogador O).
- A política ativa é atribuída dinamicamente. Se for o turno de X, avalia-se `policy_x(env, state)`; se for o turno de O, avalia-se `policy_o(env, state)`.

9. REINFORCE Agent (`reinforce.py`)

O REINFORCE é um algoritmo de gradiente de política Monte Carlo. A política stocástica é parametrizada diretamente.

Ficheiro

[reinforce.py](file:/AR_MIA/mia_rl/agents/control/reinforce.py)

9.1 Softmax Mascarado sobre Ações Disponíveis

Descrição do TODO

Implementar a função softmax restrita apenas às ações legais disponíveis no estado atual, garantindo estabilidade numérica.

Código Implementado

```
logits = (
    self.theta[available] @ phi
) # h(s,a) = theta[a]·phi(s) for each available action; shape (|available|,)
logits = (
    logits - logits.max()
) # subtract max for numerical stability (prevents exp overflow)
exp_l = np.exp(logits)
return exp_l / exp_l.sum() # normalise → probabilities sum to 1
```

Detalhes

- **Logits Lineares:** A preferência de ação (logit) para uma ação legal $a \in \mathcal{A}(s)$ é calculada por:

$$h(s, a) = \theta_a^T \phi(s)$$

No código, isto é vetorizado fatiando a matriz de parâmetros Θ (de dimensão 9×27) para conter apenas as linhas das ações legais: `self.theta[available]`. O produto matricial `@ phi` gera o vetor de preferências $h(s, \cdot)$ com dimensão $|\mathcal{A}(s)|$.

- **Estabilidade Numérica:** Valores elevados de logit podem causar estouro de capacidade (*overflow*) ao aplicar a função exponencial. Para evitar isto, subtrai-se o valor máximo dos logits:

$$h'(s, a) = h(s, a) - \max_{b \in \mathcal{A}(s)} h(s, b)$$

9.2 Atualização do Gradiente de Política (REINFORCE)

Descrição do TODO

Aplicar a regra de atualização do REINFORCE para todos os parâmetros associados às ações disponíveis no estado.

Código Implementado

```
# Escala da taxa de aprendizagem ponderada pelo fator de desconto temporal
# e o retorno G_t:
# scale = alpha * gamma^t * G_t
scale = self.alpha * (self.gamma**t) * returns[t]
for i, a in enumerate(available):
    if a == action:
        self.theta[a] += (
            scale * phi * (1.0 - probs[action_idx])
        ) # chosen: +φ·(1-π)
    else:
        self.theta[a] -= scale * phi * probs[i] # others: -φ·π
```

Fundamentos Matemáticos (Derivação do Gradiente)

A escala (scale) vai permitir aumentar ou diminuir a preferência (theta) do agente pelas ações que tomou.

10. Self-Play e Penalização no REINFORCE (reinforce_tictactoe.py)

Ficheiro

[reinforce_tictactoe.py](file:/AR_MIA/mia_rl/experiments/reinforce_tictactoe.py)

Descrição do TODO

Implementar a injeção de recompensa para o jogador perdedor na recolha de trajetórias de *self-play*.

Código Implementado

```
# Inject -1 into the loser's last trajectory step when the game ends
if done and reward == 1.0: # someone just won
    loser_traj = traj_o if player == 1 else traj_x # the other player lost
```

```
if loser_traj:
    last = loser_traj[-1]
    loser_traj[-1] = (
        last[0],
        last[1],
        last[2],
        -1.0,
    ) # overwrite r=0 → r=-1
```

Decisões

- **Problema:** No Tic-Tac-Toe, o ambiente apenas gera uma recompensa $+1.0$ para o jogador ativo que executa a jogada vitoriosa no final da partida. Contudo, sendo um jogo de soma zero, a vitória de um jogador implica necessariamente a derrota do outro. O perdedor realizou a sua última ação na jogada imediatamente anterior, onde a recompensa registada foi de 0.0 (porque o jogo ainda não tinha terminado).
- **Solução (Injeção de Recompensa):** Quando o jogo termina com vitória (`done and reward == 1.0`), identificamos quem perdeu:
 - Se o vencedor é 1 (X), o derrotado é O (`traj_o`).
 - Se o vencedor é -1 (O), o derrotado é X (`traj_x`).
- **Mutabilidade:** Como as trajetórias são guardadas como tuplos (`phi, action, available, reward`) e os tuplos em Python são imutáveis, acede-se ao último elemento da trajetória do derrotado (`last = loser_traj[-1]`), reconstrói-se o tuplo alterando o último campo (recompensa) para -1.0 , e substitui-se o elemento na lista.

11. Monte Carlo Tree Search - MCTS (`mcts.py`)

O Monte Carlo Tree Search (MCTS) é um algoritmo de planeamento baseado em simulações que não requer o treino de pesos parametrizados, utilizando o ambiente como modelo.

Ficheiro

[mcts.py](file:/AR_MIA/mia_rl/agents/planning/mcts.py)

11.1 Fase de Retropropagação (Backup)

Descrição do TODO

Implementar o método `backpropagate` da classe `MCTSNode` para atualizar as estatísticas de visitas e valores ao longo do caminho percorrido na árvore de decisão.

Código Implementado

```
def backpropagate(self, value: float) -> None:
    self.visit_count += 1
    self.value_sum += value
```

```
if self.parent is not None:
    self.parent.backpropagate(-value)
```

Detalhes

- **Visitas e Valores:** Incrementa-se a contagem de visitas em 1 e acumula-se o valor resultante da simulação em *value_sum*.
- **Alternância de Sinal (Soma Zero):** A variável *value* é passada a partir da perspectiva do jogador atual do nó. Como os nós adjacentes na árvore pertencem a jogadores opostos, o resultado que é positivo para o filho é negativo para o pai. Assim, ao propagar a informação recursivamente para o nó pai, inverte-se o sinal da recompensa (*-value*), natureza de soma zero do Tic-Tac-Toe.

11.2 Política de Rollout (Simulação)

Descrição do TODO

Implementar a simulação aleatória rápida a partir do estado atual até ao fim do jogo utilizando funções puras de cópia de tabuleiro.

Código Implementado

```
def _rollout(self, state: TicTacToeState, player: int) -> float:
    current = player
    while not _is_terminal(state):
        action = random.choice(_available(state))
        state = _apply(state, action, current)
        current = -current

    winner = _winner(state)
    if winner == player:
        return 1.0
    if winner == 0:
        return 0.0
    return -1.0
```

Detalhes

- **Funções Puras:** Para evitar mutar o estado real do tabuleiro de jogo no ambiente principal, a simulação utiliza funções auxiliares puras:
 - *_is_terminal(state)*: Verifica vitória ou empate.
 - *_available(state)*: Retorna os índices das casas vazias.
 - *_apply(state, action, player)*: Cria e retorna uma nova cópia do tuplo do tabuleiro com a jogada efetuada.
- **Lógica do Rollout:** A simulação realiza um jogo rápido selecionando ações aleatoriamente através de *random.choice(_available(state))* e alternando os jogadores (*current = -current*).

- **Retorno da Perspetiva Original:** A simulação retorna o resultado do ponto de vista do jogador que iniciou o rollout (**player**):
 - Se o vencedor for o próprio jogador, retorna $+1.0$.
 - Se for um empate, retorna 0.0 .
 - Se o vencedor for o adversário, retorna -1.0 .

12. Multi-Armed Bandits (**kbandits_incomplete.py**)

Ficheiro

[kbandits_incomplete.py](file:/AR_MIA/mia_rl/scripts/kbandits_incomplete.py)

12.1 Epsilon-Greedy Agent (**EpsilonGreedy**)

Descrição do TODO

Completar as funções **select_action** e **update** na classe **EpsilonGreedy** para implementar o algoritmo clássico de exploração e aproveitamento ϵ -greedy.

Código Implementado

```
def select_action(self):
    if np.random.rand() < self.epsilon:
        return np.random.randint(self.k)
    return np.argmax(self.Q)

def update(self, action, reward):
    self.t += 1
    self.N[action] += 1

    if self.alpha is not None:
        self.Q[action] += self.alpha * (reward - self.Q[action])
    else:
        self.Q[action] += (reward - self.Q[action]) / self.N[action]
```

Detalhes

- **Seleção de Ação ϵ -greedy:**
 - Com probabilidade ϵ (exploração), escolhe-se uma ação uniforme aleatória entre os k braços disponíveis:

$$A_t \sim \text{Uniforme}(0, k - 1)$$

- Com probabilidade $1 - \epsilon$ (aproveitamento), escolhe-se a melhor ação atual com base nas estimativas de valor $Q(a)$:

$$A_t \in \operatorname{argmax}_a Q_t(a)$$

(no NumPy, `np.argmax` desempata selecionando deterministicamente o primeiro índice de valor máximo encontrado).

- **Regra de Atualização de Valor ($Q_t(a)$):**

- Se a taxa de aprendizagem constante `self.alpha` (α) estiver ativa (útil para problemas não-estacionários onde os valores dos braços variam no tempo):

$$Q_{t+1}(A_t) \leftarrow Q_t(A_t) + \alpha [R_t - Q_t(A_t)]$$

- Caso contrário, a atualização adota a média amostral real de forma incremental, usando o número de seleções da ação $N(A_t)$:

$$Q_{t+1}(A_t) \leftarrow Q_t(A_t) + \frac{1}{N_t(A_t)} [R_t - Q_t(A_t)]$$

12.2 Upper Confidence Bound (UCB) Agent (UCB)

Descrição do TODO

Completar a função `select_action` para selecionar braços sob o critério de Limite Superior de Confiança (UCB1).

Código Implementado

```
def select_action(self):
    self.t += 1

    # ensure each action tried once
    for a in range(self.k):
        if self.N[a] == 0:
            return a

    ucb_values = self.Q + self.c * np.sqrt(np.log(self.t) / self.N)
    return np.argmax(ucb_values)
```

Detalhes

- **Inicialização dos Braços:** O UCB requer que cada ação tenha pelo menos uma amostra ($N_t(a) > 0$) para evitar divisão por zero no termo de variância. Assim, se existir algum a com $N_t(a) = 0$, o agente seleciona-o imediatamente.

12.3 Gradient Bandit Agent (GradientBandit)

Descrição do TODO

Completar as funções `select_action` e `update` da classe `GradientBandit` com base em preferências numéricas de ação $H_t(a)$ e subida de gradiente stocástica.

Código Implementado

```
def select_action(self):
    probs = self._policy()
    return np.random.choice(self.k, p=probs)

def update(self, action, reward):
    self.t += 1
    probs = self._policy()

    if self.baseline:
        self.avg_reward += (reward - self.avg_reward) / self.t
        baseline = self.avg_reward
    else:
        baseline = 0

    for a in range(self.k):
        if a == action:
            self.H[a] += self.alpha * (reward - baseline) * (1 - probs[a])
        else:
            self.H[a] -= self.alpha * (reward - baseline) * probs[a]
```

Detalhes

- **Distribuição de Ações (Softmax):** As probabilidades de seleção são calculadas a partir das preferências de ação $H_t(a)$ usando a distribuição Softmax.

Para garantir estabilidade numérica contra estouro de capacidade (overflow), subtrai-se o máximo das preferências na implementação interna de `_policy()`:

$$H'_t(a) = H_t(a) - \max_b H_t(b)$$

- **Atualização do Baseline:** Se `self.baseline` for ativado, calcula-se a média móvel incremental das recompensas obtidas ao longo do tempo:

$$\bar{R}_t \leftarrow \bar{R}_{t-1} + \frac{1}{t} [R_t - \bar{R}_{t-1}]$$

Esta média é utilizada como baseline $\bar{R}_t$ para reduzir drasticamente a variância das atualizações de gradiente sem alterar o seu valor esperado.

- **Regras de Atualização das Preferências (Gradient Ascent):**

- Para a ação selecionada A_t :

$$H_{t+1}(A_t) \leftarrow H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t))$$

- Para todas as restantes ações $a \neq A_t$:

$$H_{t+1}(a) \leftarrow H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a)$$

13. Programação Dinâmica com Gridworld (TP2_MDP_GridWorld.ipynb & Practical3_Gridworld_INCOMPLETE.ipynb)

Ficheiros

- [TP2_MDP_GridWorld.ipynb](file:/AR_MIA/mia_rl/notebooks/TP2_MDP_GridWorld.ipynb)
- [Practical3_Gridworld_INCOMPLETE.ipynb](file:/AR_MIA/mia_rl/notebooks/Practical3_Gridworld_INCOMPLETE.ipynb)

13.1 Transição de Estado no Gridworld (step)

Descrição do TODO

Completar a função `step` na classe `Gridworld` para simular as transições determinísticas na grelha 4x4.

Código Implementado

```
def step(self, state: Tuple[int,int], action: str) -> Tuple[Tuple[int,int], float, bool]:
    if self.is_terminal(state):
        return state, 0.0, True # terminal state

    delta = ACTION_TO_DELTA[action]
    next_state = (state[0] + delta[0], state[1] + delta[1])
    next_state = (max(0, min(self.n_rows - 1, next_state[0])), max(0, min(self.n_cols - 1, next_state[1])))
    reward = self.step_reward
    done_flag = self.is_terminal(next_state)

    return next_state, reward, done_flag
```

Detalhes

- Se o estado atual for terminal (por ex. a coordenada (0, 0) ou (3, 3)), a transição cessa imediatamente, devolvendo recompensa 0.0 e `done = True`.
- Caso contrário, a coordenada do próximo estado é obtida somando a variação da ação pretendida.
- Para resolver a colisão com as paredes da grelha, as coordenadas são ajustadas:

$$row_{next} = \min(\max(row + \Delta_{row}, 0), n_{rows} - 1)$$

$$col_{next} = \min(\max(col + \Delta_{col}, 0), n_{cols} - 1)$$

- Qualquer passo em estado não terminal custa uma recompensa imediata de -1.0 .

13.2 Atualização de Expectativa de Bellman

Descrição do TODO

Implementar o cálculo da atualização de valor para um estado sob uma política π (`bellman_expectation_update`).

Código Implementado

```
def bellman_expectation_update(
    env: Gridworld, V: np.ndarray, policy: Dict[Tuple[int,int],
    Dict[str,float]],
    state: Tuple[int,int], gamma: float
) -> float:
    if env.is_terminal(state):
        return 0.0

    v_new = 0.0
    for action, prob in policy[state].items():
        next_state, reward, done_flag = env.step(state, action)
        v_new += prob * (reward + gamma * V[next_state])
    return v_new
```

Detalhes

- Como o ambiente é determinístico, a probabilidade de transição $P(s', r|s, a) = 1$ para o par $(s', r) = \text{step}(s, a)$. A Equação de Expectativa de Bellman simplifica-se para:

$$V(s) = \sum_{a \in \mathcal{A}} \pi(a|s) [R(s, a) + \gamma V(s_{next})]$$

13.3 Avaliação de Política Iterativa

Descrição do TODO

Completar a função `policy_evaluation` para realizar iterações na grelha e calcular a convergência da função de valor do estado.

Código Implementado

```
def policy_evaluation(
    env: Gridworld,
    policy: Dict[Tuple[int,int], Dict[str,float]],
    gamma: float,
    theta: float = 1e-6,
    max_iters: int = 10_000,
) -> Tuple[np.ndarray, int]:
    V = zeros_V(env)

    for it in range(max_iters):
```

```

    delta = 0.0
    for state in env.states():
        v_old = V[state]
        V[state] = bellman_expectation_update(env, V, policy, state,
gamma)
        delta = max(delta, abs(V[state] - v_old))

    if delta < theta:
        return V, it + 1

    return V, max_iters

```

Detalhes

- O algoritmo aplica sucessivas atualizações *in-place* sobre o vetor V .
- Para garantir a convergência formal, mede-se o erro máximo absoluto:

$$\Delta \leftarrow \max(\Delta, |V(s) - V_{\text{old}}(s)|)$$

O loop de iterações termina quando $\Delta < \theta$ (limiar de precisão numérica).

13.4 Extração de Política Gulosa

Descrição do TODO

Implementar a extração da política ótima a partir da função de valor de estado (lookahead de um passo) nas funções `greedy_policy_from_V` e `policy_improvement`.

Código Implementado

```

# Em TP2_MDP_GridWorld.ipynb (greedy_policy_from_V):
best_a = None
best_q = float('-inf')
for action in ACTIONS:
    next_state, reward, done_flag = env.step(s, action)
    q = reward + gamma * V[next_state]
    if q > best_q:
        best_q = q
        best_a = action
pi_greedy[s] = best_a

# Em Practical3_Gridworld_INCOMPLETE.ipynb (policy_improvement):
best_a = None
best_q = -np.inf
for a in ACTIONS:
    ns, r, done = env.step(s, a)
    q = r + gamma * V[ns[0], ns[1]]
    if q > best_q:
        best_q = q

```

```

        best_a = a
    new_policy_actions[s] = best_a
    if old_policy_actions and old_policy_actions[s] != new_policy_actions[s]:
        stable = False

```

Detalhes

- A extração da política gulosa calcula os valores de ação $q(s, a)$ e seleciona a ação de maior valor:

$$\pi'(s) \leftarrow \operatorname{argmax}_{a \in \mathcal{A}} q(s, a) = \operatorname{argmax}_{a \in \mathcal{A}} [R(s, a) + \gamma V(s_{next})]$$

- No método **policy_improvement**, se a ação gulosa $\pi'(s)$ diferir da ação da política antiga $\pi(s)$, o indicador **stable** é atualizado para **False**, forçando o algoritmo de iteração de política a continuar por mais uma etapa.

13.5 Exercícios Teóricos e Experimentais

Exercício A — Impacto do Fator de Desconto γ

- $\gamma = 0.5$ (**Agente Míope**): Recompensas de curto prazo dominam a função de valor. O valor absoluto acumulado de penalidades é fortemente atenuado, resultando em magnitudes de estado V^* baixas (menos negativas). O agente prioriza caminhos curtos e imediatos.
- $\gamma = 0.99$ (**Agente Paciente**): O agente dá quase a mesma relevância a recompensas no futuro distante que às imediatas. A magnitude absoluta da função de valor de otimalidade cresce consideravelmente (mais negativa), e o algoritmo necessita de muito mais iterações para convergir, dado que a constante de contração do erro γ está muito próxima de 1.

Exercício B — Adição de Célula de Armadilha (Trap Cell) e Comparação VI vs PI

- Impacto da Armadilha em (1, 2) com $r = -10.0$:** A política ótima altera-se radicalmente para contornar a célula de penalização. O agente prefere fazer caminhos mais longos na grelha (com custos acumulados de -1.0 por passo) para evitar colidir ou transitar pela célula penalizada.
- Comparação de Iterações (Value Iteration vs Policy Iteration):**
 - Value Iteration (VI):** O operador de otimalidade de Bellman ($\max_a$) faz com que a informação de perigo e recompensa propague-se de forma direta, convergindo em pouquíssimas iterações externas.
 - Policy Iteration (PI):** Converge em poucas iterações externas de melhoria de política, mas cada uma delas exige uma avaliação de política completa interna (policy evaluation), a qual requer dezenas de iterações na grelha. Por conseguinte, VI é globalmente mais leve computacionalmente na grelha clássica de Gridworld.

Exercício C — Ambiente Estocástico (Transições com Deslizamento)

- Dinâmica Estocástica:** Com probabilidade 0.8 o agente move-se na direção pretendida, e com probabilidade 0.1 para a esquerda ou direita (deslizamento).

- **Impacto em V^* :** Os valores dos estados tornam-se globalmente mais negativos devido ao risco estatístico de sofrer colisões não intencionais com as paredes ou de ser "empurrado" para longe do objetivo.
- **Impacto na Política Gulosa:** O agente adota uma política de maior margem de segurança. Evita rotas próximas a cantos estreitos e armadilhas onde um deslize probabilístico causaria uma grande penalização.

14. Jack's Car Rental ([Practical3_CarRental_INCOMPLETE.ipynb](#))

Ficheiro

[Practical3_CarRental_INCOMPLETE.ipynb]

(file:/AR_MIA/mia_rl/notebooks/Practical3_CarRental_INCOMPLETE.ipynb)

14.1 Função de Valor de Ação Expectável ($q(s, a)$) via `q_from_v`

Descrição do TODO

Implementar o retorno expectável de valor $q(s, a)$ de acordo com o modelo de aluguer e custos de transporte do MDP.

Código Implementado

```
def q_from_v(mdp: CarRentalMDP, V: np.ndarray, s: Tuple[int,int], a: int,
gamma: float) -> float:
    p_next_1, p_next_2, exp_revenue = mdp.expected_transition(s, a)
    expected_next_value = 0.0
    for n1_next, p1 in enumerate(p_next_1):
        for n2_next, p2 in enumerate(p_next_2):
            expected_next_value += p1 * p2 * V[n1_next, n2_next]
    reward = exp_revenue - mdp.params.cost_per_moved * abs(a)

    return reward + gamma * expected_next_value
```

Detalhes

- O estado é representado por (n_1, n_2) (número de carros em cada posto). A ação a é o número de carros movidos do posto 1 para o posto 2 (valores negativos indicam deslocamento inverso).
- O custo de movimentação de carros é linear: $C_{mov} = \text{cost_per_moved} \times |a|$.
- Como o número de alugueres e retornos segue distribuições de Poisson independentes, a transição gera distribuições marginais de probabilidade para a quantidade de carros em cada posto, `p_next_1` e `p_next_2`.
- O valor esperado do próximo estado-valor é computado usando a independência das marginais:

$$\mathbb{E}[V(s')] = \sum_{n'_1=0}^{M_1} \sum_{n'_2=0}^{M_2} P(n'_1 | s_1, a) P(n'_2 | s_2, a) V(n'_1, n'_2)$$

- O retorno expectável total $q(s, a)$ une a recompensa imediata (receita esperada menos o custo do transporte) ao termo de bootstrap descontado:

$$q(s, a) = \mathcal{R}_{esperado} - C_{mov} + \gamma \mathbb{E}[V(s')]$$

14.2 Avaliação de Política Iterativa

Descrição do TODO

Completar a função `policy_evaluation` para estimar V^π sob a política atual determinística de Jack's Car Rental.

Código Implementado

```
for state in mdp.states():
    v_old = V[state]
    V[state] = bellman_expectation_backup_v(mdp, V_old, state, policy,
gamma)
    delta = max(delta, abs(V[state] - v_old))
```

Detalhes

- O método itera por todos os estados possíveis (n_1, n_2) da grelha de carros (limite de 20 carros por posto, ou seja, $21 \times 21 = 441$ estados).
- O cálculo do bootstrap `bellman_expectation_backup_v` chama a função `q_from_v` usando a ação definida pela política `policy[state]`.

14.3 Melhoria de Política Gulosa

Descrição do TODO

Completar a função `policy_improvement` para selecionar a ação gulosa π' em cada estado e verificar a estabilidade do processo.

Código Implementado

```
best_a = None
best_q = float('-inf')

for a in mdp.possible_actions(s):
    q = q_from_v(mdp, V, s, a, gamma)
    if q > best_q:
```

```

        best_q = q
        best_a = a
    new_policy[s] = best_a
    if old_policy and old_policy[s] != new_policy[s]:
        stable = False

```

Detalhes

- A melhor ação gulosa maximiza a expectativa:

$$\pi'(s) \leftarrow \operatorname{argmax}_{a \in \mathcal{A}(s)} q(s, a)$$

- O algoritmo compara as ações da nova política com `old_policy`. Se houver qualquer divergência, a política ainda não atingiu a otimalidade de Bellman, definindo `stable = False`.

14.4 Iteração de Política

Descrição do TODO

Completar a lógica principal da função `policy_iteration` para intercalar avaliação de política e melhoria de política de forma iterativa.

Código Implementado

```

V, eval_iters = policy_evaluation(mdp, policy, gamma, theta)
history.append((V.copy(), policy.copy(), eval_iters))
policy, stable = policy_improvement(mdp, V, policy, gamma)
if stable:
    print(f"Policy iteration converged at outer iteration {outer}.")
    break

```

Detalhes

- A iteração de política (Policy Iteration) converge de forma robusta.
- A cada ciclo, a avaliação de política calcula a função de valor exata de V^π que, posteriormente, serve de base para melhorar a política localmente em direção à otimalidade.

14.5 Iteração de Valor

Descrição do TODO

Completar o passo principal de `value_iteration` aplicando o operador de otimalidade de Bellman para atualizar o valor dos estados diretamente.

Código Implementado


```
for state in mdp.states():
    v_old = V[state]
    V[state] = bellman_optimality_backup_v(mdp, V_old, state, gamma)
    delta = max(delta, abs(V[state] - v_old))
```

Detalhes

- A cada iteração de valor (Value Iteration), em vez de seguir uma política fixa, o algoritmo atualiza o valor de cada estado visando o melhor retorno possível sob o operador de Bellman:

$$V(s) \leftarrow \max_{a \in \mathcal{A}(s)} q(s, a)$$

onde `bellman_optimality_backup_v` seleciona o maior $q(s, a)$ avaliado em todas as ações válidas de movimentação de carros.

- Ao convergir (quando o erro máximo entre estados Δ fica abaixo de θ), a política ótima é obtida fazendo um último lookahead guloso sobre a função de valor ótima convergida V^* .